COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE (EE-2003)

ASSIGNMENT # 3

ID: _____ **NAME: _____** **SECTION: ____**

Read the Instructions Carefully

- Assigned number is given to you in excel sheet provided with assignment consider it as decimal

FOR EXAMPLE: Assigned Number if your assigned number is **7823** where A0 is first digit of assigned number.

Fill in your roll numbers according to the specifications provided below.

	Assign Digit 0	Assign Digit 1	Assign Digit 2	Assign Digit 3
Short for Assigned Digit	**A0**	**A1**	**A2**	**A3**
Write Assigned Number Digit By Digit	**7**	**8**	**2**	**3**
	Assigned Byte 0		**Assigned Byte 1**	
Short for Assigned BYTE	**B0**		**B1**	
Byte in DECIMAL	**78**		**23**	
Convert byte to HEX B0H, B1H	**4E**		**17**	
Convert byte OCTAL B0Q, B1Q	**116**		**27**	
Convert byte BINARY B0B, B1B	**0100 1110**		**0001 0111**	
	Assigned WORD			
Short for Assigned WORD	**W**			
WORD in DECIMAL(W)	**7823**			
Convert WORD to HEX (WH)	**1E8F**			
Convert WORD OCTAL (WQ)	**17 217**			
Convert byte BINARY (WB)	**0001 1110 1000 1111**			
Assigned double WORD in HEXA (DD H)	**78237823H**			

EXAMPLE: Name is **HAMZA DAUD**

- *If your name starts with **MUHAMMAD** kindly use your second name*

	ZERO CHARACTER OF YOUR NAME	FIRST CHARACTER OF YOUR NAME	SECOND CHARACTER OF YOUR NAME	THIRD CHARACTER OF YOUR NAME	FOURTH CHARACTER OF YOUR NAME	FIFTH CHARACTER OF YOUR NAME	SIXTH CHARACTER OF YOUR NAME
Short for CHARACTER	**C0**	**C1**	**C2**	**C3**	**C4**	**C5**	**C6**
YOUR NAME CHARACTER BY CHARACTER	**H**	**A**	**M**	**Z**	**A**	**D**	**A**

1. Consider the following code and fill in the given registers and memory accordingly after each step?

;**NOTE**

- **ADC** ax,bx ;AX+BX+CF(carry flag) ;ADC is add through carry
- You are supposed to run all 16 iterations
- Update **multiplicand,Result** and **multiplier** after every iteration

```
.data
    multiplicand dd 0WH        ;Hexa of assigned word
    multiplier dw 005AAH
    result dd 0

.code
main PROC
mov eax,0

mov cl,16                      ;initialize bit count to 16
mov dx, multiplier             ;initialize bit mask

checkbit:
   shr dx,1
   jnc noadd                   ;skip addition if no carry

   mov ax, word ptr[multiplicand] ;mov LSW of multiplicand to ax
   add word ptr[result],ax        ;add LSW of multiplicand to result
   mov bx, word ptr[multiplicand+2];mov MSW of multiplicand to bx
   adc word ptr[result+2],bx  ;add MSW of multiplicand to result
noadd:
   shl word ptr[multiplicand],1    ;shift LSW multiplicand to left
   rcl word ptr[multiplicand+2],1
                                  ;rotate MSW of multiplicand to left
Loop checkbit
```

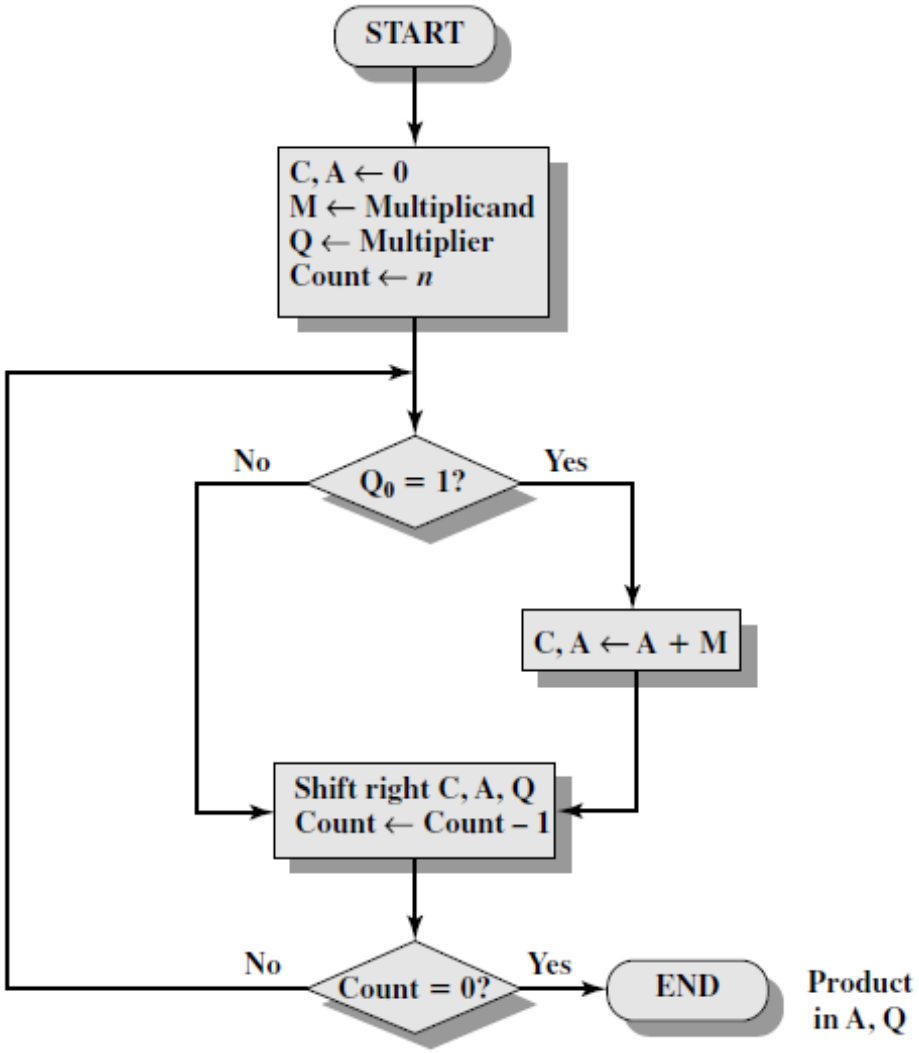
CF	BX	CF	AX	DX	CF		
	Multiplicand + 2		**Multiplicand**	**Multiplier**		**Result+2**	**Result**

CF	BX	CF	AX	DX	CF		
	Multiplicand + 2		Multiplicand	Multiplier		Result+2	Result

2. Modify and rewrite the above given code for following data declaration?

```
.data
  multiplicand DQ DH  ;value of Assigned doubleword in hexa
  multiplier DW WH     ;Assigned word in hexa
  result DQ 0          ;result of the multiplication
```

3. Perform unsigned binary multiplication using following given flow chart? Use flow chart to generate code as well
NOTE: Your computer width is 8-bit, **Multiplicand** is $(\mathbf{B0B})_2$ of the assigned number and **Multiplier** is $(\mathbf{B1B})_2$ of the assigned number



C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
				Add
				Shift
C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
				Add
				Shift
C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
				Add

				Shif t

C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
				Add
				Shif t

C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
				Add
				Shif t

C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
				Add
				Shif t

C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
				Add
				Shif t

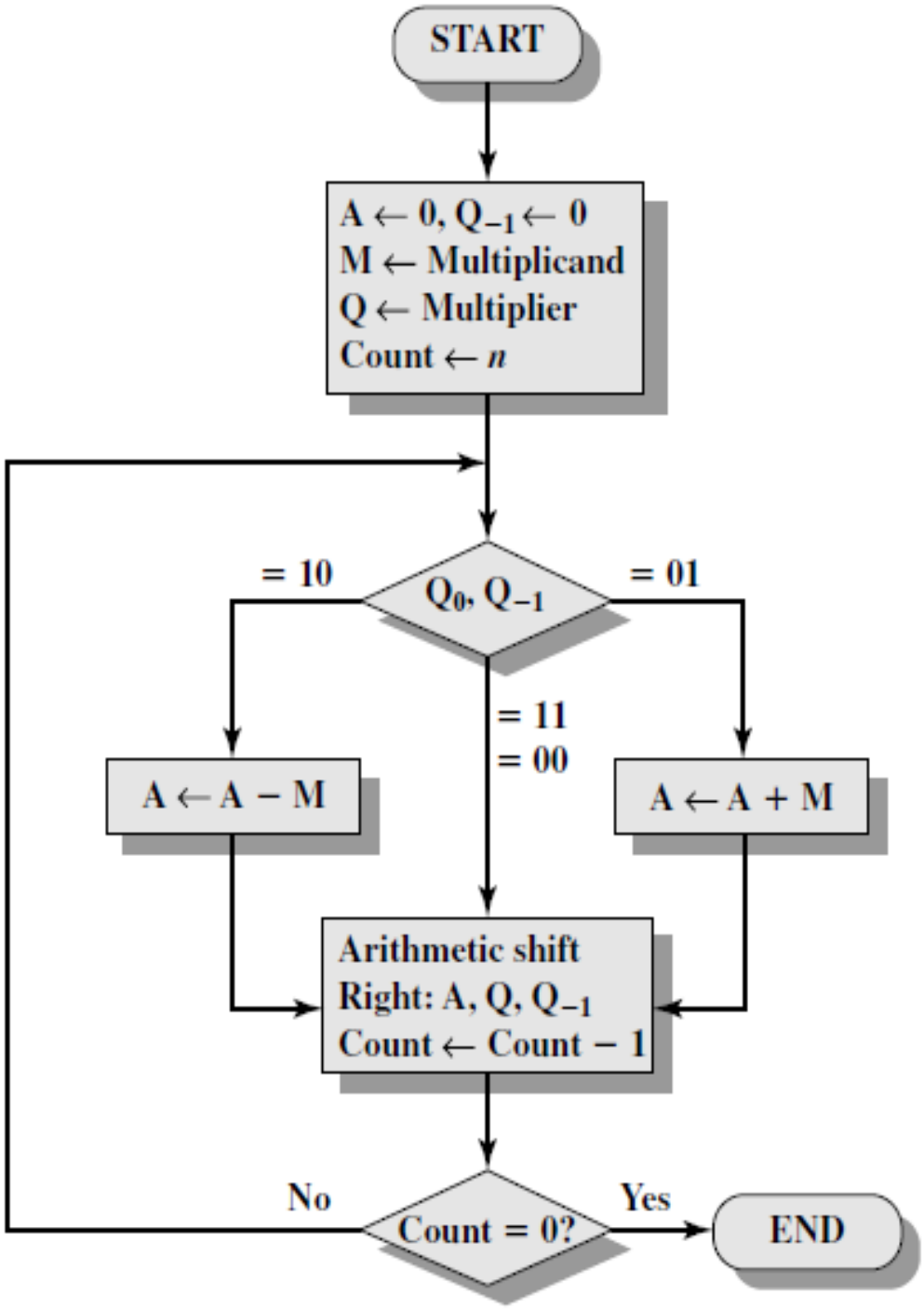
C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
				Add
				Shif t

C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
				Add
				Shif t

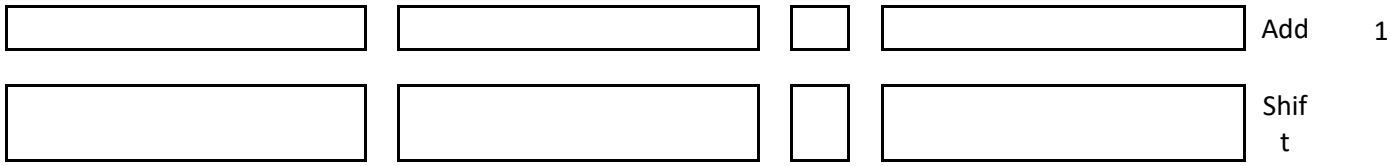
C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
				Add
				Shif t

4. Perform Multiplication using Booth's algorithm? Use flow chart to generate code as well

NOTE: Your computer width is 10-bit, **Multiplicand** is 2's complement of **B1H** assigned number (Means it's a signed and negative number), **Multiplier** is **0A5H?**

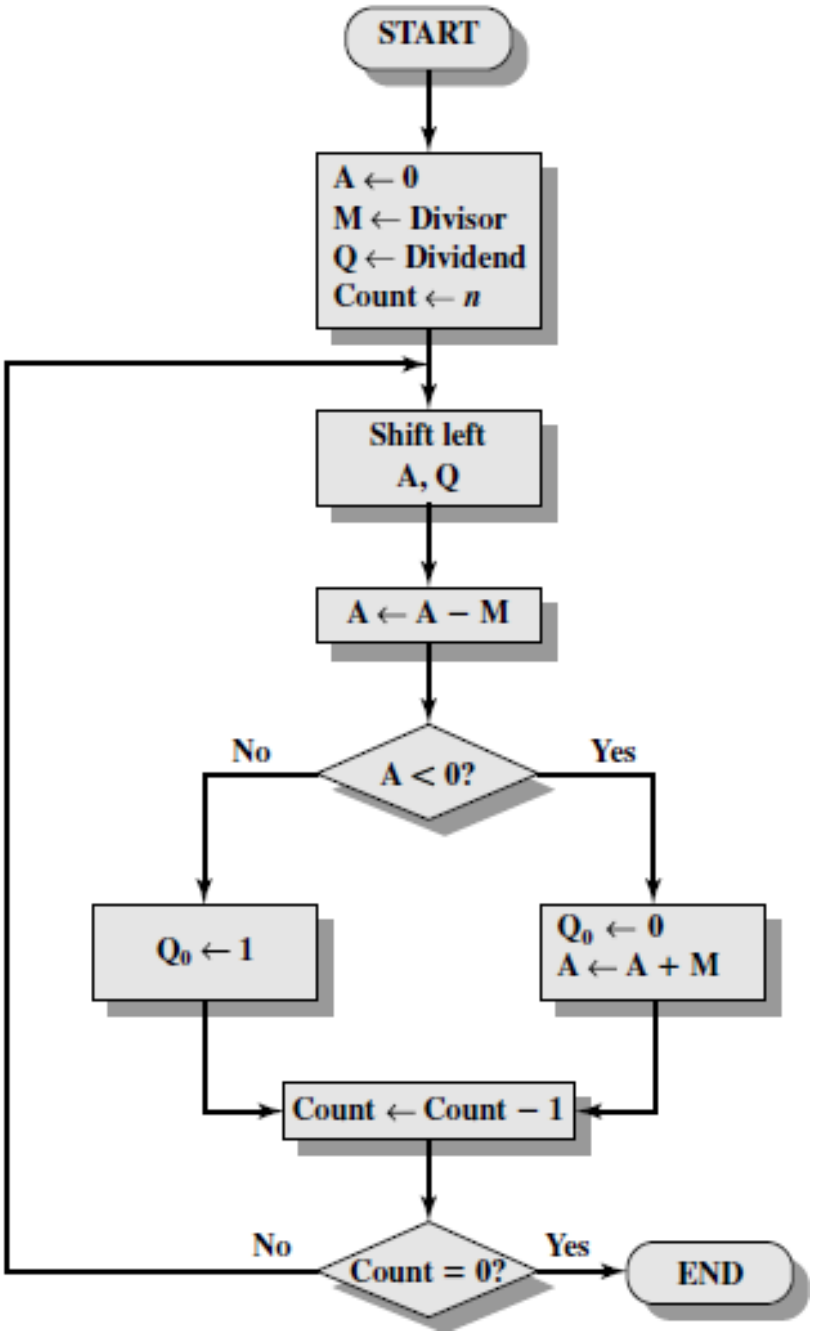


A (ACCUMULATOR)	Q (MULTIPLIER)	Q -1	M (MULTIPLICAND)		
				Add	10
				Shift	
				Add	9
				Shift	
				Add	8
				Shift	
				Add	7
				Shift	
				Add	6
				Shift	
				Add	5
				Shift	
				Add	4
				Shift	
				Add	3
				Shift	
				Add	2
				Shift	


Add
1
Shif
t

5. Perform Unsigned binary division. Use flow chart to generate code as well

NOTE: Your computer width is 8-bit, where dividend **B1H** of assigned number, **Divisor** is **003H**



A/ACCUMULATOR	Q/DIVIDEND	COUNT	OPERATION

6. Write a program that Left shift your character of your name three times to left and treat it as array. **Write final result in HEX.**

```
nameSTRING db 'CO', 'C1', 'C2', 'C3', 'C4', 'C5', 'C6', 'C7', 'C8', 'C9'    ; C0
s first character of your name
     ; C9 is tenth character of your name
```

7. Apply the following piece of code and update the registers accordingly.

```
mov al, 05fh

shr al, 1
```

REGISTER (AL)									CF

```
mov al, 0f5h

sar al, 1
```

REGISTER (AL)									CF

```
mov al, 05fh

rol al, 1
```

REGISTER (AL)									CF

```
mov al, 0f5h

rcl al,1
```

REGISTER (AL)									CF

8. You are working on the development of an encryption process for a secure communication protocol for a highly sensitive application. As part of the encryption process, the system needs to perform a series of bitwise operations on the provided data. Your task is to **design and implement an assembly program that can do these series of transformations on a 32-bit integer number mDATA.**

Suppose an 16-bit number **mFLAGS**, you have to transform the data by applying the series of following 2-bitwise operations on **mDATA** according to the specific bit value in **mFLAGS** as defined in the operation description provided below:

Operation #	Operation Description
1	**Double-Precision Shift Left:** Perform the **Double-Precision Shift Left**. The shift must be performed X times. The number X must be extracted from the **mFLAGS** using its last 4 bits (LSB). The destination must be **mDATA**, the source must be **mFLAGS**, and the count is X.
2	**Bitwise RIGHT Rotation:** Perform the **right bitwise** rotation. The rotations must be performed X times. The number X must be extracted from the **mFLAGS** using its first 5 bits. Ensure that the rotation is circular, meaning bits shifted out from one end are rotated back to the other end. Note: Not allowed to use any predefined rotate instruction like (ROL,ROR etc.)

Note: You have to generate 16-bit number **mFLAGS** using your **Roll Number** excluding the character and the first digit. For example if the number is **22i-9986** the mFLAGS must be **29986**.

9. ASCII and Unpacked Decimal Arithmetic

A. You must dry-run the following program and fill in the registers after each iteration and execution.
Note: For DECIMAL_TWO your number will be equal to
1 A_1 A_2 A_0 A_1 A_2A_3 A_0 A_1 A_2A_3 A_0 A_1 A_2A_3
For example 182 7823 7823 7823

```
.data
          decimal_one BYTE "985123456789765"
          decimal_two BYTE  Roll number as above
          sum BYTE (SIZEOF decimal_one + 1) DUP(0),0
.code
main Proc
mov esi,SIZEOF decimal_one - 1
mov edi,SIZEOF decimal_one
mov ecx,SIZEOF decimal_one
mov bh,0                                ; set carry value to zero
L1: mov ah,0                            ; clear AH before addition
mov al,decimal_one[esi]                 ; get the first digit
add al,bh                               ; add the previous carry
aaa                                          ; adjust the sum (AH =
carry)
mov bh,ah                               ; save the carry in carry1
or bh,30h                               ; convert it to ASCII
add al,decimal_two[esi]                 ; add the second digit
aaa                                     ; adjust the sum (AH =carry)
or bh,ah                                ; OR the carry with carry1
or bh,30h                               ; convert it to ASCII
or al,30h                               ; convert AL back to ASCII
mov sum[edi],al                         ; save it in the sum
dec esi                                 ; back up one digit
dec edi
loop L1
mov sum[edi],bh                         ; save last carry digit

mov bx,0
```

Iteration 01:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 02:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 03:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 04:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 05:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 06:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 07:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 08:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 09:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 10:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 11:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 12:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 13:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 14:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

Iteration 15:

AL (add al,decimal_two[esi])	**AL (After AAA)**

BH (or bh,30h)	**SUM (15-digits)**

B. Convert the above code to Perform subtraction **DECIMAL_TWO** and **DECIMAL_ONE**.

Note: Use AAS instructions.

```
.data
decimal_one BYTE "985123456789765"
decimal_two BYTE  Roll number as above
subtract BYTE (SIZEOF decimal_one+1) DUP(0),0
```

10. Write a procedure that performs simple encryption by rotating each plaintext byte a varying number of positions in different directions. For example, in the following array that represents the encryption key, a **negative** value indicates a rotation to the **left** and a **positive** value indicates a rotation to the **right**. The integer in each position indicates the **magnitude** of the rotation:

key BYTE -5, 2, 3, 0, -1, -4, 3, -2, 4, 6

Your procedure should loop through a plaintext message and align the key to the first 10 bytes of the message. Rotate each plaintext byte by the amount indicated by its matching key array value. Then, align the key to the next 10 bytes of the message and repeat the process. Write a program that tests your encryption procedure.

11. Write a procedure BinarytoASC which will convert a binary integer into an ASCII binary string. The procedure must display the result. Write a program which tests your procedure as well.

For example, a binary Integer 00100011 will be converted into a string (in ASCIIs) “00100011”.

Hint: Use SHL.

12. Convert the following C++ program to assembly language using ***recursive stack procedures***.

```cpp
// Function to calculate the nth Fibonacci number using recursion
int nthFibonacci(int n){

    // Base case: if n is 0 or 1, return n
    if (n <= 1){
        return n;
    }

    // Recursive case: sum of the two preceding Fibonacci numbers
    return nthFibonacci(n - 1) + nthFibonacci(n - 2);
}
```

13. Convert the following C++ program to assembly language using ***recursive stack procedures.***

```
int findMaxRec(int A[], int n)
{
    // if n = 0 means whole array has been traversed
    if (n == 1)
        return A[0];
    return max(A[n-1], findMaxRec(A, n-1));
}
```

14. You are tasked to use **Sign Extension Instructions** for this question.
For dividing signed numbers using the **IDIV** instructions, in the past, you have used movsx and movzx commands to extend the sign of the dividend in a larger register, as demonstrated:

```
.data
byteval sbyte -48
.code
movsx ax, byteval
mov bl, 5
idiv bl
```

Write an Assembly Language code that uses **Sign Extension Instructions (CBW, CWD, and CDQ)** to perform the following divisions:

- **a)** Divide the signed byte -125 by 5
- **b)** Divide the signed word FFB1h by 7
- **c)** Divide the signed dword FFFFB1B0h by 6

Provide screenshots for each output (quotient and remainder).

Note: You **must** use CBW, CWD, and CDQ. Do **NOT** use movsx.